Auction e-Commerce System

Team 9: Code2Cash

EECS 4413: Deliverable 3

Winter 2026

Project Github Repository:

https://github.com/PeytonHislop/EECS4413Auction_e-Commerce_System

Ravneet Deol - 217270067

Syed Mustafa Jamal - 217761651

Anton Caushaj - 219468024

Peyton Hislop - 217342957

Document Change Control Revision History

Version	Date	Author(s)	Summary of Changes
1.0	Jan 25, 2026	Ravneet Deol	Created the document outline
1.1	Feb 2, 2026	Ravneet Deol	Update to use cases, architecture - services, introduction - references, major design decisions,
1.2	Feb 2, 2026	Syed Mustafa Jamal	Edited document formatting, added UC's related to auction service, added base/structure for auction diagrams, added comments for group clarifications
1.3	Feb 3, 2026	Ravneet Deol	Update to use cases, major design decisions - architectural patterns chosen, test-driven development
1.4	Feb 4, 2026	Anton Caushaj	Completion of the Project Purpose/Overview description, updates to use cases, and the addition of final drafts of the Catalogue Activity and Sequence diagrams. Major design decisions added: Catalogue Test Driven Development, Catalogue Architecture. Added requested items to the product backlog.

1.5	Feb 4, 2026	Ravneet Deol	Update to test-driven development, use cases, major design decisions, product backlog,
1.6	Feb 5, 2026	Ravneet Deol	Sequence diagram, Activity diagram, Architecture - Interfaces, Major design decisions - architectural patterns chosen
1.7	Feb 5, 2026	Peyton Hislop	Sequence, activity diagram, payment service use cases
1.8	Feb 6, 2026	Peyton Hislop	Added Services, interfaces and test cases
1.9	Feb 6, 2026	Syed Mustafa Jamal	Added sequence, activity & architecture & UC test-cases, added architecture C, added acknowledgements, formatted document and adjusted sizes of charts for better clarity
2.0	Feb 6, 2026	Ravneet Deol	Updated architectural patterns chosen, architectural patterns not chosen, completed Gantt diagram, completed component diagram
2.1	March 06, 2026	Ravneet Deol	Added an implementation updates section. Updated test cases, services, and interfaces. Added to the AI disclosure section.

2.2	March 6, 2026	Anton Caushaj	Test case updates, implementation updates for the Catalogue service and project design patterns chosen/not chosen.
2.3	March 6, 2026	Syed Mustafa Jamal	Added implementation updates for Auction service, test case updates.
2.4	March 6, 2026	Peyton Hislop	Added implementation updates for Payment service. Test case updates
3.1	March 28, 2026	Syed Mustafa Jamal	Created leaderboard-service, cleaned up auction service test cases
3.2	March 28, 2026	Ravneet Deol	Worked on the REACT frontend and developed the file/folder structure.
3.3	March 30, 2026	Syed Mustafa Jamal	Practice demo'd Auction and Leaderboard services with the front end, added more features to leaderboard, implemented HATEOAS for both services, implemented live bid and live auction via websocket.
3.4	March 30, 2026	Peyton Hislop	Added implementation and test updates for Payment service added postman test script
3.5	March 30, 2026	Anton Caushaj	Implementation updates for Catalogue backend/frontend, JMeter test/report

3.6	March 31, 2026	Ravneet Deol	Bugged fixed issues related to the catalogue service endpoint and frontend.
3.7	March 31, 2026	Ravneet Deol	Updated readme.md
3.8	March 31, 2026	Everyone	Group meeting to run a test demo and finish document
3.9	April 1, 2026	Syed Mustafa Jamal	Fixed some problems with Leaderboard and Auction, Added docker support to the git

Table of Contents

Document Change Control Revision History.....2
Table of Contents.....5
Introduction.....6
Major Design Decisions.....7
Modularization Criteria:.....7
Architectural patterns chosen:.....7
Architectural pattern not chosen:.....8
Implementation Updates Deliverable 3.....9
IAM implementation summary.....9
Frontend implementation summary.....9
Catalogue implementation summary.....9
Auction implementation summary.....10
Payment implementation summary.....11
Explanation of a Distinguishable Feature:.....12
Leaderboard Service Implementation Summary:.....12
Performance Report.....14
Implementation Updates Deliverable 2.....15
IAM implementation summary.....15
Catalogue implementation summary.....15
Auction implementation summary.....16
Payment implementation summary.....17
Use Cases.....18
Identity and Authorization Management (IAM) Service Use Cases:.....18
Auction Service (AS) UCs:.....18
Catalogue Service UCs:.....19
Payment Service UCs:.....19
Sequence Diagrams.....20
Activity Diagrams.....23
Component Diagram.....26
Architecture.....27
Services:.....27
Interfaces:.....28
Activities Plan, Product Backlog, and Sprint Backlog.....35
Product backlog:.....35
GANTT diagram:.....37
Test Driven Development.....38
Ravneet Test Cases.....38
Anton Test Cases.....46
Mustafa Test Cases.....49
Peyton Test Cases.....63
Acknowledgements.....68

Introduction

Purpose: The general objective of this project is to design and implement a robust e-commerce auction system using open-source web-based frameworks. More specifically, the project focuses on a Forward Auction where an item is placed for sale by a seller and a buyer must enter increasingly strict bids until the time runs out (at which point the item will no longer be available). Furthermore, this project integrates architectural goals beyond simple programming constructs, such as intended microservice deployments and a modular design through the separation of business logic, request operations, and front-end implementations.

Overview: This software application helps fix the problem of having a secure online auction in real time at a distance from seller to buyer. It essentially creates a marketplace where all users can access the same list of items for available bidding, track live bidding wars, and securely process payments for won items. The overall system requirements mandate a three-tier architecture:

Front-end: An HTML/CSS-based interface for user interaction.

Middle-tier: Framework-based layer (e.g. SpringBoot), which allows for swift relay requests and API communication.

Back-end: Independent services containing business logic and Database integration/connectivity.

This is a microservices architecture. Each service runs in its own process, meaning that IAM, Catalogue, Auction and Payment have separate databases and they communicate with each other through open standards (e.g., HTTP/REST) to fulfill application requirements.

Overall, the system must support core functionalities, including User Identity Management (Sign-Up/Sign-In), Catalogue Management (Search and Display), Auctioning Engines (validating bids), and Payment Processing.

References:

- EECS 4413 Project Description: Building e-Commerce Systems. York University, 2026.
- Forward Auction. Wikipedia. (Referenced in Project Description)
- *Gantt Chart*. Wikipedia. (Referenced in Deliverable 1 Instructions)

Major Design Decisions

Modularization Criteria:

- High cohesion
    - Our system's design focuses on keeping each module focused on one primary function. This allows each service to have a clear purpose. For example, the Identity and Authorization Management Service focuses on only identity and authentication, password reset, and retrieval of a user's profile. This improves quality because each service has a narrow focus, is able to be tested independently, and can be changed without affecting unrelated functionality.
- Low coupling
    - Our system's design minimizes the dependencies that exist between modules by limiting how each service interacts with one another. Services will communicate with each other via a clear set of REST endpoints that other services can call. The gateway service will act as a single entry point for clients to keep the internal service structure isolated from the front-end. Additionally, each service will own its own database and will not be able to write directly to another service's database.

Architectural patterns chosen:

1. We will adopt a three-tier architecture to separate concerns.
    - Front-end subsystem - HTML/CSS/JS pages
    - Middle subsystem - Gateway service will receive UI requests and orchestrate calls to backend services
    - Back-end subsystem - Contains business logic and data persistence
2. Microservices-oriented modular design
    - Each service will run independently and have its own datastore. Supports parallel team development. Each service will be able to be tested independently and evolve separately.
3. API Gateway/Facade
    - Gateway functions as the UI's stable entry point. Will enforce token checks. The UI will stay simple, and the system will avoid exposing internal details to the front-end.
4. DAO Pattern
    - Each service will use DAOs to isolate how they interact with persistence logic. Will keep SQL details away from business logic. Will allow persistence to be changed more easily in the future if needed without having to rewrite domain logic.

5. Pub-Sub
    - The methods through which users will receive real-time bid updates. Will allow all users to see the current highest bid without requiring constant page refreshes.
6. Dependency Injection
    - Spring's @Autowired keyword throughout allows the promotion of loose coupling, testability (referring to @MockBean), and the overall lifecycle management of the different components.

Architectural pattern not chosen:

- Decided against the usage of a message broker such as Kafka at the moment. The pros of using such a pattern would improve decoupling and scaling for our project, but it would add complexity. Instead, the project will focus on using REST and WebSocket for real-time updates. In the future, if we believe the benefits outweigh the cons for deliverables 2 and 3, we may reconsider our thoughts on this choice.
- We also decided against the usage of the Circuit Breaker pattern. Omitting the use of the Breaker pattern is an intentional architectural decision that prioritizes simplicity and learning. Since the services are running on a local host with no actual network failures, and the services have a hard dependency on each other, implementing Circuit Breakers would only add unnecessary complexity without solving the problems that arise in the completion of the project. The fail-fast behaviour enables immediate debugging of issues rather than masking them with complex logic, making it easier to understand how the microservices interact/depend on each other.

Implementation Updates Deliverable 3

IAM implementation summary

- Tech stack: Java 21, Spring Boot, Spring Web, Spring Security, Spring Data JPA, Jakarta Validation, JJWT, and SQLite for persistent storage.
- This service is responsible for identity and authorization management for the e-commerce platform, including account creation, login, token validation, role checks, password reset, and user profile retrieval for downstream services.
- The IAM service now uses SQLite instead of H2, giving the service persistent local storage.
- Gateway forwards authentication-related client requests to IAM, while protected services such as Catalogue and Auction rely on the JWT issued by IAM
- The IAM service has test coverage for successful and failed sign-up, successful and failed sign-in, valid and invalid token validation, authorized and unauthorized role checks, missing parameter handling, user profile retrieval, forgot-password token generation, password reset, and successful login after password reset.

Frontend implementation summary

- Tech Stack: React 18.3.1, Build Tool: Vite 6.0.7, Styling: Plain CSS with modular component styles.
- Architecture Pattern: Modular Service-Oriented Frontend, the codebase mirrors the microservices backend, with each module owning its own pages, components, and API calls.
- API Client Features: Automatic JSON parsing and error handling, Bearer token injection for authenticated requests, Query parameter builder (?keyword=laptop&status=ACTIVE), Unified error format with meaningful messages, Support for GET, POST, PUT, PATCH, DELETE methods
- All 4 services are implemented based on pages implemented with React, different pages are implemented based on necessary use case completion.

Catalogue implementation summary

- Tech Stack: Spring Boot 3.2.0, Java 21, Spring Web / Data JPA / Validation / HATEOAS, H2 Database 2.2.220, RestTemplate (inter-service HTTP)
- SQLite with schema auto-initialization on startup, separate database file
- Port: 8083
- What's implemented: UC-CAT-2 (Item Detail), UC-CAT-2.1 (Item Search with filtering), and UC-CAT-7 (Create Auction Item): full static catalogue management with authentication/authorization enforcement.
- New Implementation: The Catalogue Service implements Richardson Maturity Level 3 REST (HATEOAS) using Spring HATEOAS. Every item returned by the REST API includes hypermedia links for navigation. The ItemModelAssembler wraps each Item entity in an EntityModel that contains _links to the item's self URL and the collection URL. Collections are returned as CollectionModel<EntityModel<Item>> with self-referential links. This enables clients to discover available actions dynamically without hardcoding URLs. Inter-service authentication is handled via RestTemplate HTTP calls to the IAM Service (port 8081). When a SELLER attempts to create an item via POST /api/catalogue/items, the controller extracts the JWT Bearer token from the Authorization header, calls IAMService.validateToken() using POST to /auth/validate, then calls authorizeRole() using GET with a query parameter to /auth/authorize?requiredRole=SELLER. Only after both validations pass is the item persisted with the authenticated user's UUID as sellerId.

Auction implementation summary

- Tech stack: Spring Boot 3.2.0, Java 17, Spring Web / JDBC / Validation / WebSocket (STOMP), SQLite 3.44.1, WebFlux (inter-service HTTP), JdbcTemplate
- Database: SQLite with schema auto-initialization on startup, separate database file
- Port: 8082

- What's implemented: UC1-AS through UC4-AS — full auction and bid management lifecycle, plus real-time bid notification (C8), and the new Leaderboard Service (UC-LB-01 through UC-LB-04)
- New Implementation: A STOMP-over-WebSocket broker was added to the Auction Service. After any successful bid is placed via the REST API, BidServiceImpl publishes a BidUpdateMessage to the topic /topic/auctions/{auctionId}/bid-update using Spring's SimpMessagingTemplate. The React frontend subscribes to this topic for each open auction page via the useBidUpdates hook. Any connected client — whether on the auction detail page or the active auctions list — receives the new highest bid, updated bidder ID, and current bid count instantly without requiring a page refresh. The WebSocket endpoint is exposed at /ws with SockJS fallback support for broad browser compatibility.

Payment implementation summary

- Tech stack: Java 17, Maven 3.9.12 (build tool), Spring Boot 3.1.5, Port: 8084
- SQLite with automatic entity mapping, persistence of payments and orders and schema generation
- Gateway Integration: Payment Service does not directly communicate with other services the gateway gets required data before sending the request
- Port 8084
- One api endpoint POST /api/payments/process that handles receipt generation based on auction id. Where card info is entered
- What's implemented: UC-PAY-1 and UC-PAY-2 payment page. The user adds the credit card number, the name on the card, the expiration date and the security code. Once payment is cleared, the winning bidder is served with a receipt and shipment information. The page provides the total amount paid as well as the number of days shipping will take.

Explanation of a Distinguishable Feature:

The Real-Time Weekly Leaderboard Service was introduced to enhance the auction platform by giving users a clearer sense of how their activity compares to others. While the core system allowed bidding, it lacked visibility into overall competition, which this feature addresses. The leaderboard tracks live bid activity and updates frequently, making it feel like a natural extension of the existing real-time system rather than a separate add-on. Unlike other components that were required from the start, this feature was independently designed by the team, making it a unique contribution to the project. It focuses only on current weekly activity, resetting regularly so that rankings remain relevant and do not accumulate unnecessary data over time. The service provides several ways to view information, such as overall rankings, individual bidder activity, and past weekly data, all of which are reflected through simple frontend pages. From a user perspective, this adds a sense of competition and engagement, while from a system perspective, it shows how different parts of the application can work together to process and display live data. Testing confirmed that updates were reflected correctly after each bid, user-specific data was accurate, and outdated entries were properly removed, with all related tests passing successfully.

Leaderboard Service Implementation Summary:

Leaderboard Service implementation summary (Deliverable 3)

- Tech stack: Spring Boot 3.2.0, Java 21, Spring Web / JPA / HATEOAS, H2 in-memory database, Lombok
- Database: H2 in-memory (jdbc:h2:mem:leaderboarddb) — resets on service restart, suitable for weekly tracking demo
- Port: 8085

What's implemented (UC-LB-01 through UC-LB-04):

UC-LB-01 — Weekly Leaderboard: GET /api/leaderboard returns the top 10 highest individual bids placed during the current Monday–Sunday window, ranked by bid amount with ties broken by earliest bid time.

UC-LB-02 — Weekly Statistics: GET /api/leaderboard/stats returns aggregated statistics for the current week including total bid count, highest bid, average bid, and total bid value across all tracked entries.

UC-LB-03 — Bidder Stats Lookup: GET /api/leaderboard/bidder/{bidderId} returns all bids placed by a specific bidder in the current week, sorted descending by amount, along with a computed highest bid summary.

UC-LB-04 — Historical Leaderboard: GET /api/leaderboard/week/{year}/{week} returns the top-10 leaderboard for any past ISO week by year and week number. GET /api/leaderboard/highest?period=WEEK and GET /api/leaderboard/top-bidders?period=WEEK&limit=N provide additional period-based queries

- How other services use it: The Auction Service calls the Leaderboard Service's internal addBidEntry() after each successful bid via ServiceClient. The frontend accesses the Leaderboard Service directly via a Vite proxy (/leaderboard-api → port 8085) rather than through the gateway, since the leaderboard is a distinguished feature service not part of the core gateway routing.
- Design patterns implemented:
    - Repository Pattern — Spring Data JPA LeaderboardRepository with custom JPQL queries for time-range filtering and top-N selection
    - DTO Pattern — LeaderboardEntryResponse and TopBidderResponse separate persistence entities from API contracts
    - Service Layer Pattern — LeaderboardService encapsulates all week-range calculation, ranking, and cleanup logic
    - Scheduled Tasks — @Scheduled for automatic weekly data cleanup

Performance Report

Testing Tool: Apache JMeter 5.6.3

Testing Scope: IAM, Auction, Catalogue, Payment Services

Initial Bug Count (v_0): 500

Key Findings:

Overall System Failure Rate: 3.2%

Most Reliable Service: Catalogue (0.8% failure rate)

Highest Failure Service: Payment (7.1% failure rate)

Performance Report: Response Time vs. Arrival Rate

We utilized JMeter to simulate concurrent user traffic across our microservices to measure how system response times degraded as the arrival rate increased. The test spanned 4 hours and generated 38,916 total requests.

Test Results by Arrival Rate

Light Load (10-15 concurrent users): The system easily handled the baseline traffic. Average response times ranged from 28ms (Catalogue) to 112ms (Payment).

Medium Load (50-75 concurrent users): The system represented typical production load. Response times remained acceptable for the Catalogue (51ms) and IAM (89ms) services, but the Payment service began exceeding SLA limits (234ms).

Heavy Load (100-150 concurrent users): The system reached its stress point. While the Catalogue service remained relatively stable (98ms average), the IAM and Auction services degraded significantly (187ms and 156ms, respectively). The Payment service became severely bottlenecked, averaging 389ms with a 95th percentile response time of 1,024ms.

Failure Rates

Across the 38,916 requests, the system exhibited an overall failure rate of 2.67%.

Service-Specific Failure Breakdown

Catalogue Service: 0.74% failure rate (Most stable), IAM Service: 3.61% failure rate, Auction Service: 4.20% failure rate, Payment Service: 7.16% failure rate (Least stable).

CPU Testing Time Estimation

As per the testing parameters, we assume an initial defect count of v0 = 500. Based on our testing duration (16 total CPU-hours across 4 services) and the discovery of 22 primary defects, we derived a failure rate proportionality constant of approx Φ=0.0003106. To estimate the CPU testing time (T) required to sequentially find and eliminate all 500 bugs, we calculate the sum of the time intervals between failures using the harmonic series:

$$T = \frac{1}{0.0003106} \sum_{i=1}^{500} \frac{1}{i}$$

The harmonic number for 500 (H{500}) is approximately 6.792.

T= (1/0.003106) * 6.792 = About 21,867 hours needed.

To theoretically eliminate all 500 bugs and reach a 100% defect-free state, the system requires an estimated 21,867 CPU-hours of continuous testing. This would translate to about 136 calendar weeks of continuous QA.

Deployment (Docker)

Added deployment via Docker Desktop. The project should run with `docker compose up --build`.

Implementation Updates Deliverable 2

IAM implementation summary

- Tech stack: Spring Boot 4.0.3, Java 21, Spring Web/Security/JPA/Validation, H2, JJWT
- What's implemented: UC-IAM-01 to UC-IAM-06
- How other services use it: pass JWT in Authorization: Bearer <token>; use /auth/validate and /auth/authorize; use /users/{userId} for shipping profile.

Catalogue implementation summary

- Tech stack: Java 21 (LTS), Maven 3.9.12 (build tool), Spring Boot 3.2.0, JUnit 5, Spring Boot Test, JaCoCo, Port: 8083
- Implemented: UC-CAT-2, UC-CAT-2.1, UC-CAT-7
- How other services use it: The Gateway service acts as a proxy, where client requests are forwarded from /api/items/* using the RestClient, using the port 8083. The Catalogue service validates authentication by calling the IAM service at 8081 using RestTemplate to check the JWT tokens and verify the seller roles before actually allowing the item creation.
- Design Patterns implemented: **Microservice architecture** - Services communicate via synchronous HTTP/REST calls. **Facade Pattern** - CatalogueFacade provides an interface to complex subsystems, such as logic filtering, repository interactions, encapsulations of data

such as date calculations, and repository interactions from the controller layer. **Dependency Injection** - Spring's @Autowired keyword throughout allows the promotion of loose coupling and testability (referring to @MockBean).

Auction implementation summary

- Tech stack: Spring Boot 3.2.0, Java 17, Spring Web/JDBC/Validation, SQLite 3.44.1, WebFlux (for inter-service HTTP calls), JdbcTemplate
- Database: : SQLite with schema auto-initialization on startup, separate database file
- Port: 8082
- What's implemented: UC1-AS through UC4-AS - auction and bid management lifecycle
- How other services use it:
    - Gateway Service routes all /api/auctions/* requests to this service (port 8080 → 8082), Public endpoints (GET active auctions, bid history) require no authentication, Protected endpoints (POST auction, POST bid) require JWT in Authorization: Bearer <token> header, Service validates tokens and roles via IAM service endpoints /auth/validate and /auth/authorize, Automatic auction closure via scheduler (runs every 60 seconds, checks for expired auctions), Calls Catalogue service to verify item exists before creating auction, Calls Payment service when auction closes with a winner
- Design patterns implemented:
    - DAO Pattern - Separates database access from business logic
    - DTO Pattern - Clean API contracts with Jakarta validation
    - Service Layer Pattern - Business logic isolated from controllers
    - MVC Pattern - Clear separation of Controller → Service → DAO
- Key features:
    - RESTful API with 11 endpoints, JWT authentication and role-based authorization

- Real-time bid validation (must exceed current highest), Automatic auction closure and winner determination, Comprehensive error handling and validation, Database persistence with SQLite

Payment implementation summary

- Tech stack: Java 17, Maven 3.9.12 (build tool), Spring Boot 3.1.5, Port: 8084
- SQLite with automatic entity mapping, persistence of payments and orders and schema generation
- Gateway Integration: Payment Service does not directly communicate with other services the gateway gets required data before sending the request
- Port 8084
- One api endpoint POST /api/payments/process that handles receipt generation based on auction id. Where card info is entered
- IAM Integration the system uses JWT-based authentication Payment requests must contain Authorization: Bearer <JWT>
- This service uses all other services through the gateway to get data and generate receipt
- Design patterns implemented: MVC Pattern - Clear separation of Controller → Service → Repository → Database. Repository Pattern uses repository classes/interfaces to isolate database access (PaymentRepository, OrderRepository). DTO Pattern used to separate objects for request/response transfer instead of exposing entities directly. Facade Pattern (PaymentClient) acts like facade over more complicated subsystem. API Gateway Pattern validating the JWT with IAM and obtaining data from other services through the gateway this gives cleaner inter-service communication.

- What’s implemented: UC-PAY-1 and UC-PAY-2 - payment handling and receipt generation

Use Cases

<u>Identity and Authorization Management (IAM) Service Use Cases:</u>

UC-IAM-01: Sign-Up

- Description: A user creates a new account

UC-IAM-02: Sign-In

- Description: A user's username and password are verified to start a session

UC-IAM-03: Forgot Password/Reset Password

- Description: A user is able to reset their login credentials if forgotten

UC-IAM-04: Validate Session/Token

- Description: Every protected action will confirm that the user is logged in.

UC-IAM-05: Authorize Role-Based Actions

- Description: Decides if a user is able to perform an action based on the scope of actions they're limited to in their role.

UC-IAM-06: Provide User Profile For Payment

- Description: The payment page must show the correct user's information (shipping address).

<u>Auction Service (AS) UCs:</u>

UC1-AS: Create Auction Listing

Description: A seller creates a new auction listing for an item they want to sell

UC2-AS: Place Bid on Auction

Description: A buyer places a bid on an active auction

UC3-AS: Close Auction and Determine Winner

Description: The system automatically closes an auction when the time expires and determines the winner

UC4-AS: View Active Auctions

Description: A user browses all currently active auctions

Catalogue Service UCs:

UC-CAT-2: Browse the Catalogue of Auctioned Items

UC-CAT-2.1: Item Search

Description: Once logged in, the user provides a keyword, and a list of items up for auction (or currently being auctioned) matching the keywords are displayed.

Responsibility: The service queries the database for item descriptions matching the keyword and filters out items that are no longer active.

UC-CAT-7: Auction Item (Static Catalogue)

Description: A seller uploads information to the system to list an item for sale.

This use case handles the Content Creation (what is being sold), while UC1-AS handles the Market Initialization (opening the bidding floor).

Responsibility: The Catalogue Service validates the seller's input (Description, Forward Auction Type, Duration, Shipping Price) and calculates the absolute End Date by adding the duration to the current timestamp. It persists this static record to the Catalogue Database.

Payment Service UCs:

UC-PAY-1: Payment Page

Description: Once the winner selects the “Pay Now” button, the winning bidder is served with a page to pay with the user's address from the database, the item price and shipping cost. The user adds the credit card number, the name on the card, the expiration date and the security code.

UC-PAY-2: Receipt Page and Shipping details

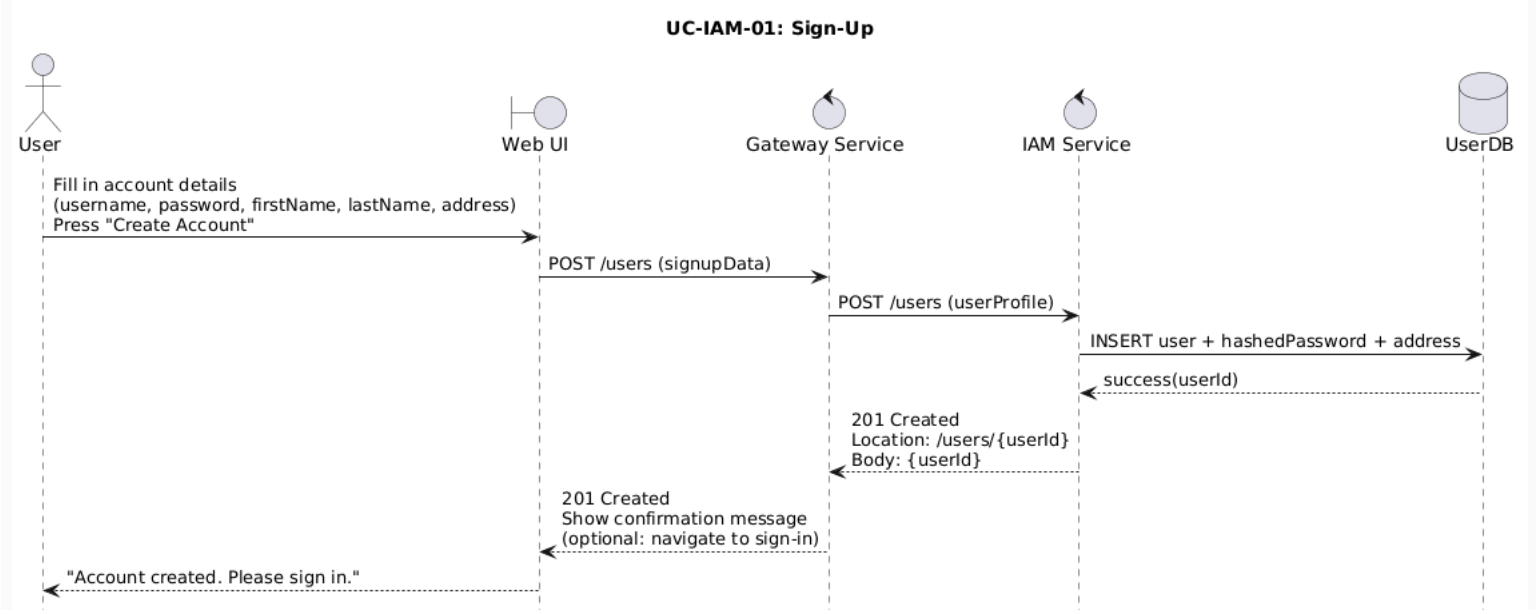
Description: Once payment is cleared, the winning bidder is served with a receipt and shipment information. The page provides the total amount paid as well as the number of days shipping will take.

Sequence Diagrams

Identity and Authorization Management Service Sequence Diagram for UC-IAM-01:

<u>Legend</u>

Stick figure (Actor)	External user that interacts with the system
Participant header (Web UI, Gateway Service, IAM Services, UserDB)	A system component that sends and receives messages
Vertical dashed line	The time span that a participant exists for during interactions.
Horizontal solid arrow	A request being sent
Horizontal dashed arrow	A response being returned
Message label text	Description for actions being performed



Auction Service

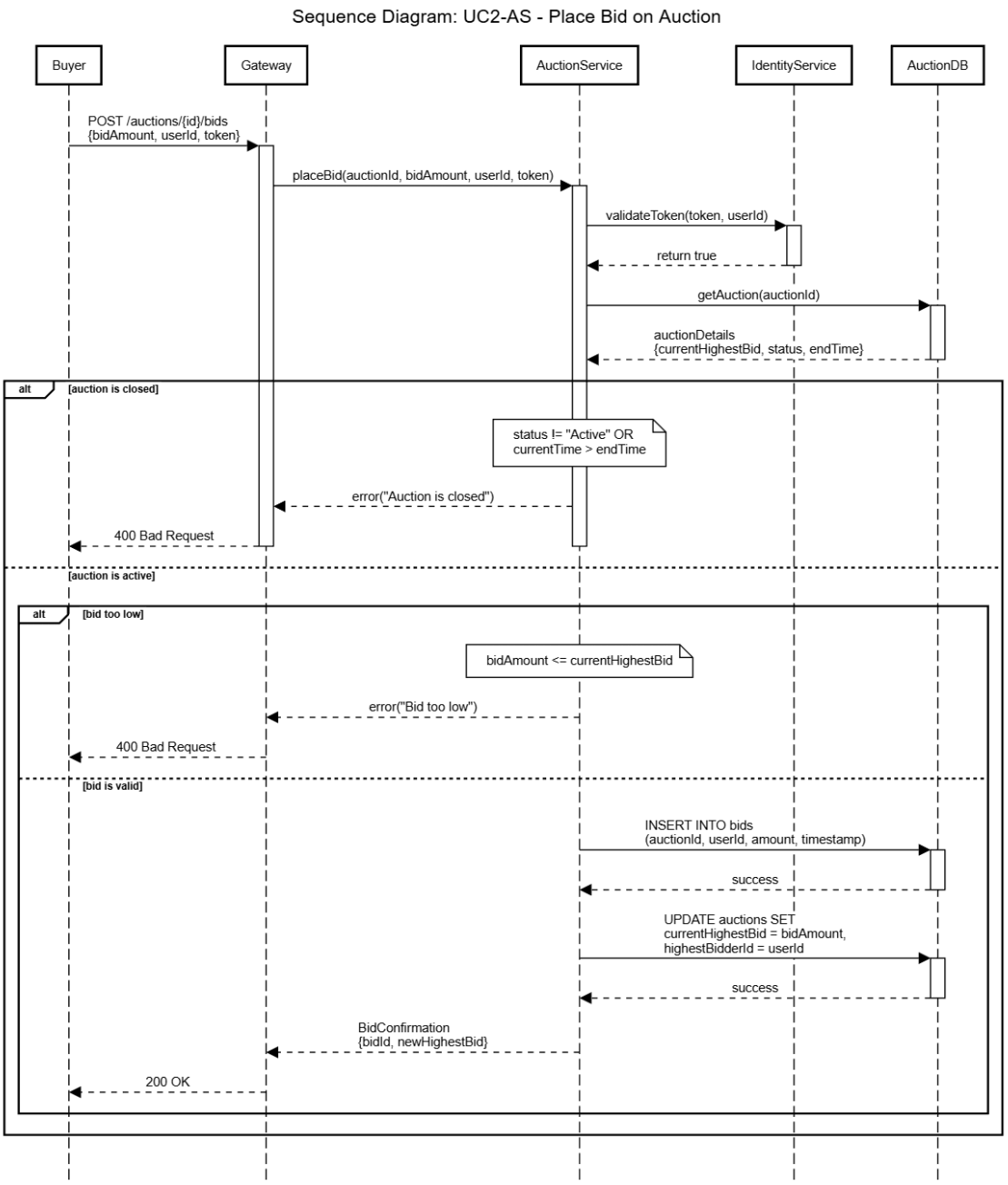
Sequence

Diagram,

Use Case: UC6

- Place Bid on

Auction



Legend: (for the Auction diagram above, Catalogue diagram and Payment diagrams below)

Solid Arrow (→): Synchronous method call or request

Dashed Arrow (- - →): Return value or response

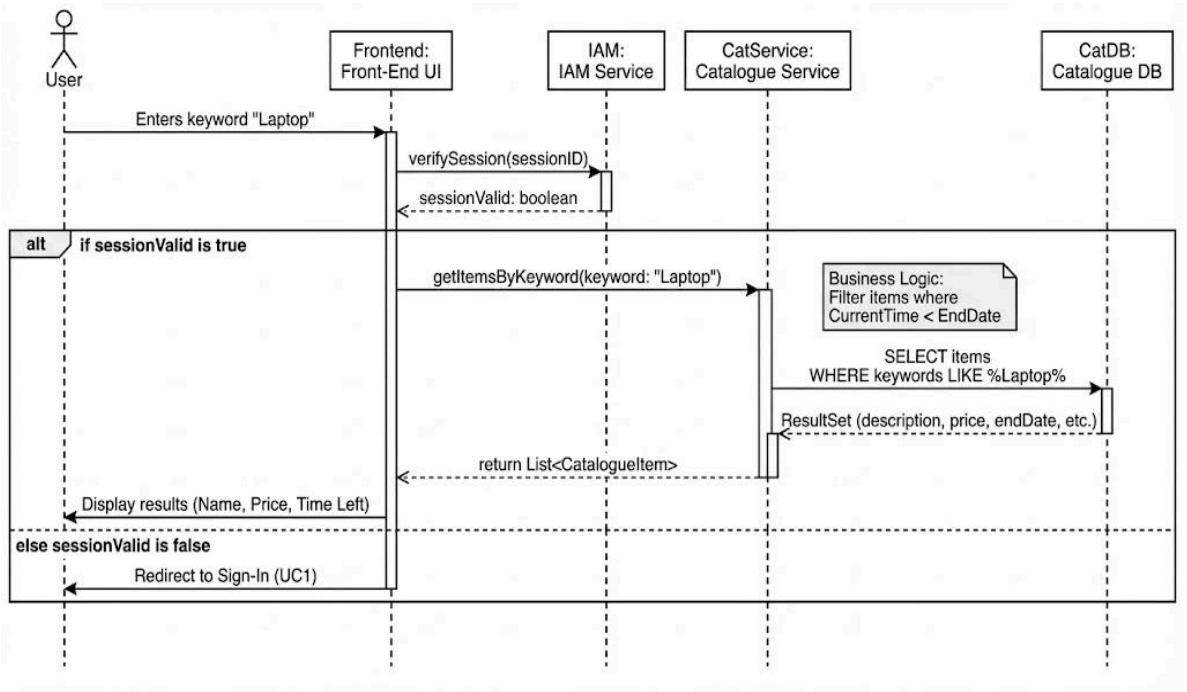
Vertical Rectangle: Activation period (component is processing)

Vertical Dashed Line: Lifeline (component exists)

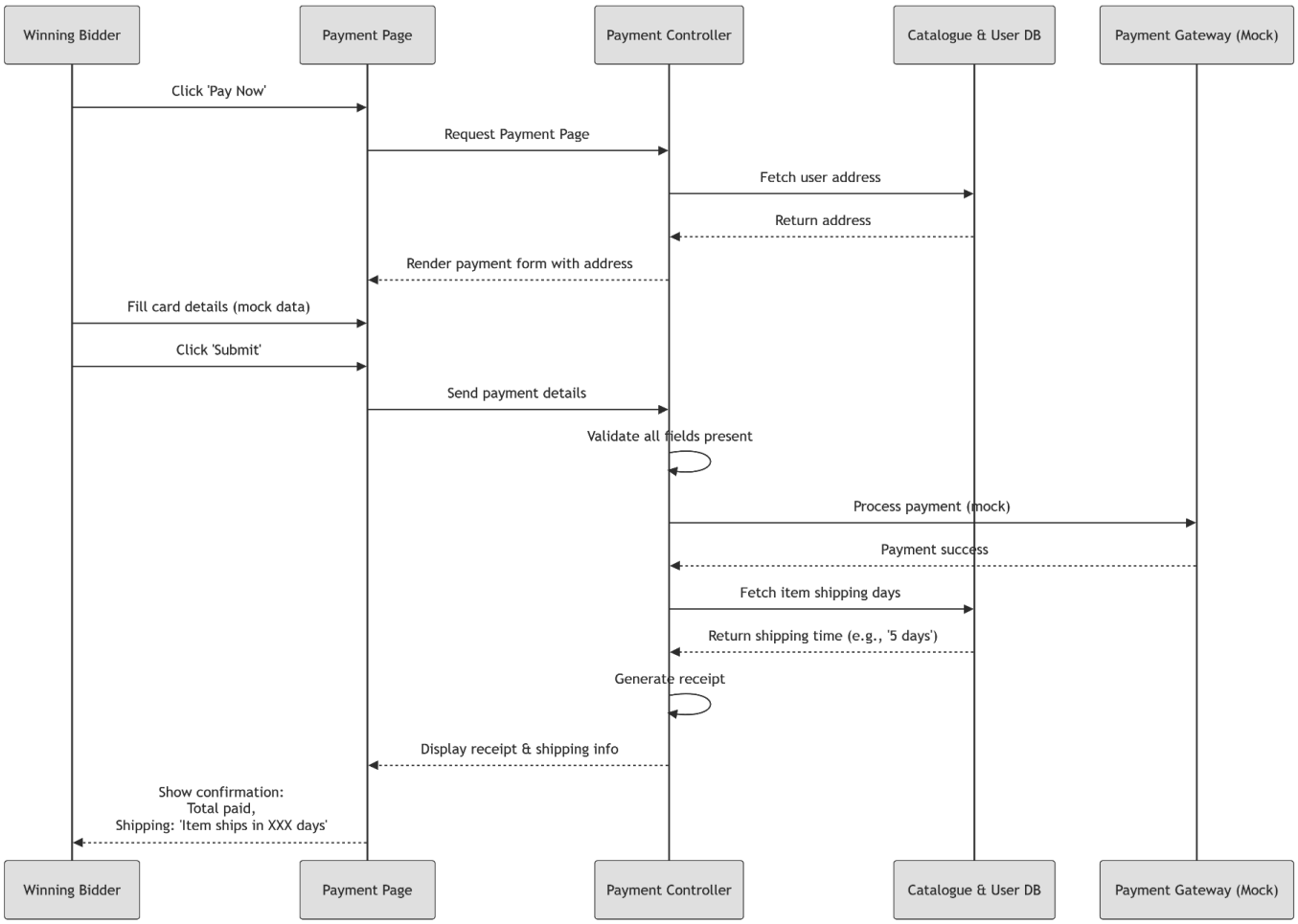
Alt Box: Conditional logic with alternative paths

Note Box: Explanation of conditions or business logic

Sequence Diagram for Use Case UC-CAT-2.1 - Item Search:



Sequence Diagram for Use Case UC-PAY-1, UC-PAY-2



Activity Diagrams

Identity and Authorization Management Service Components Diagram for UC-IAM-01:

<u>Legend (Applies for IAM service and catalogue service activity diagrams)</u>

Rounded rectangle - Action/Activity step.

Swimlane - What service is performing the steps.

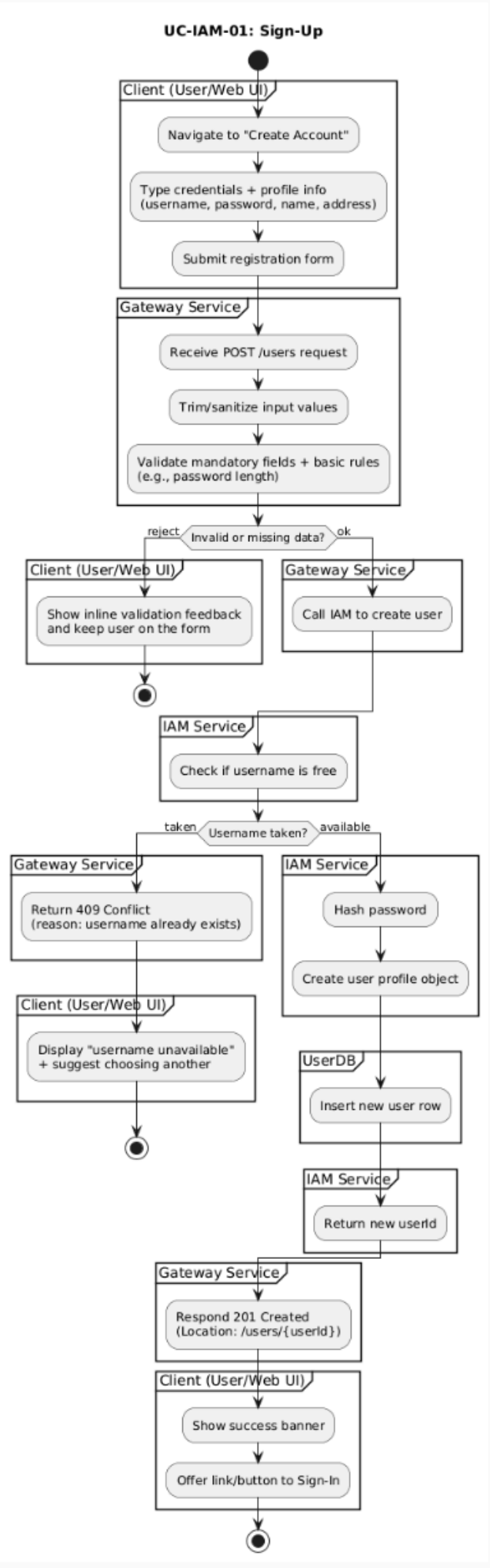
Solid arrowed line - Control flow.

Diamond: A branch point that picks a path based on a condition.

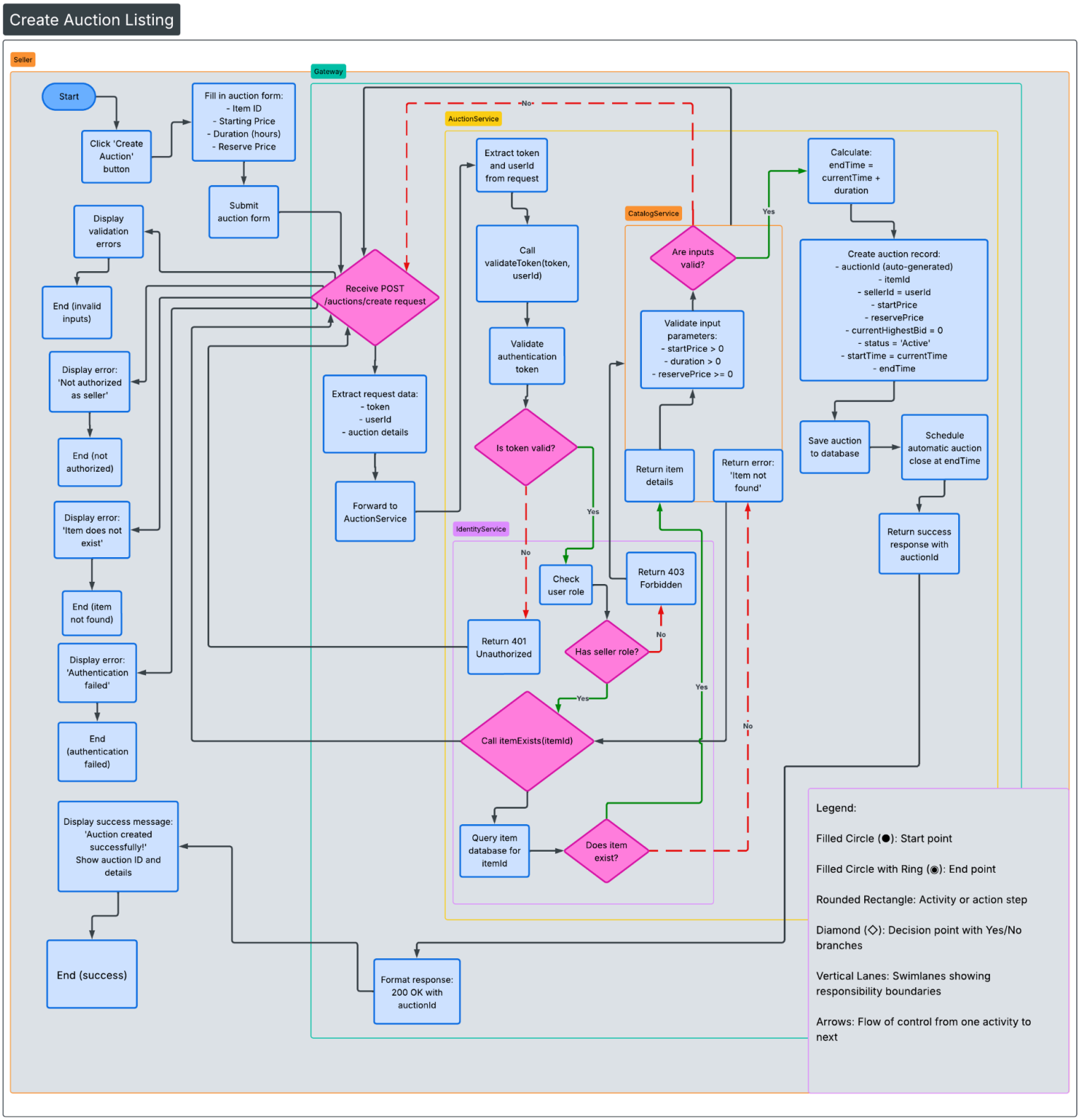
Decision labels: Condition outcomes that select outgoing branches.

Black circle - Where the process starts.

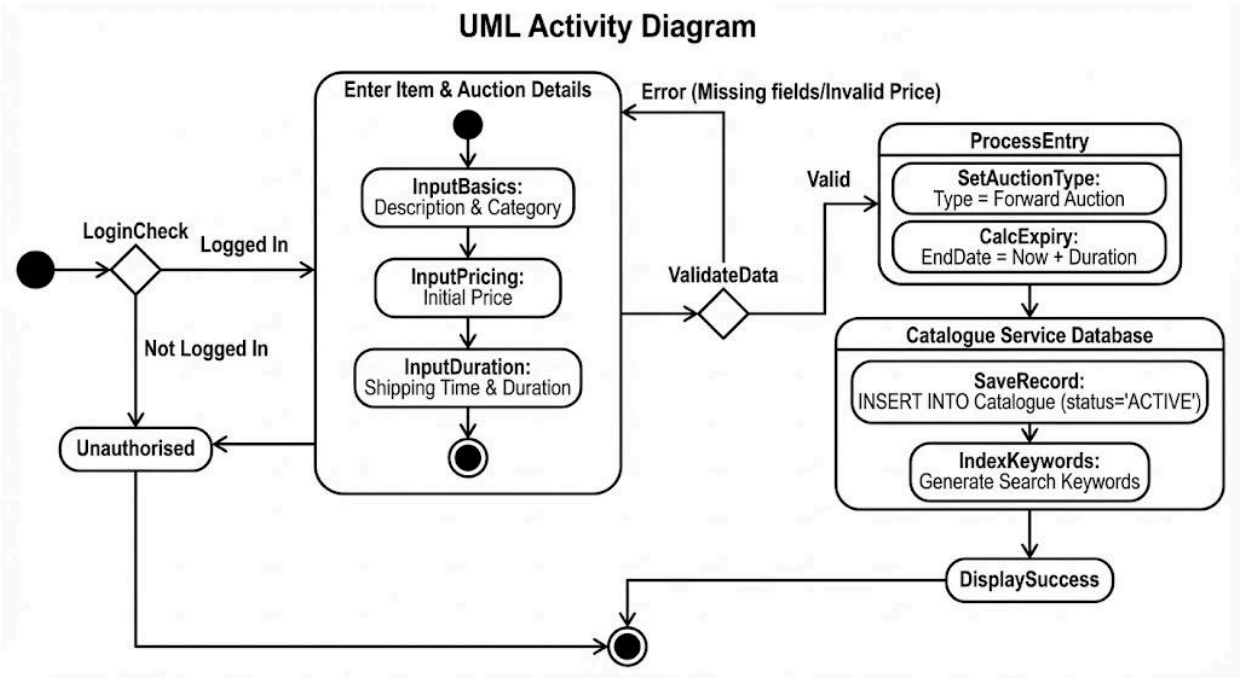
Target circle - Where the process terminates.



Auction Service Activity Diagram, Use Case: UC5 - Create Auction Listing



Activity Diagram for UC-CAT-7 - Auction Item (Static Catalogue) Listing:



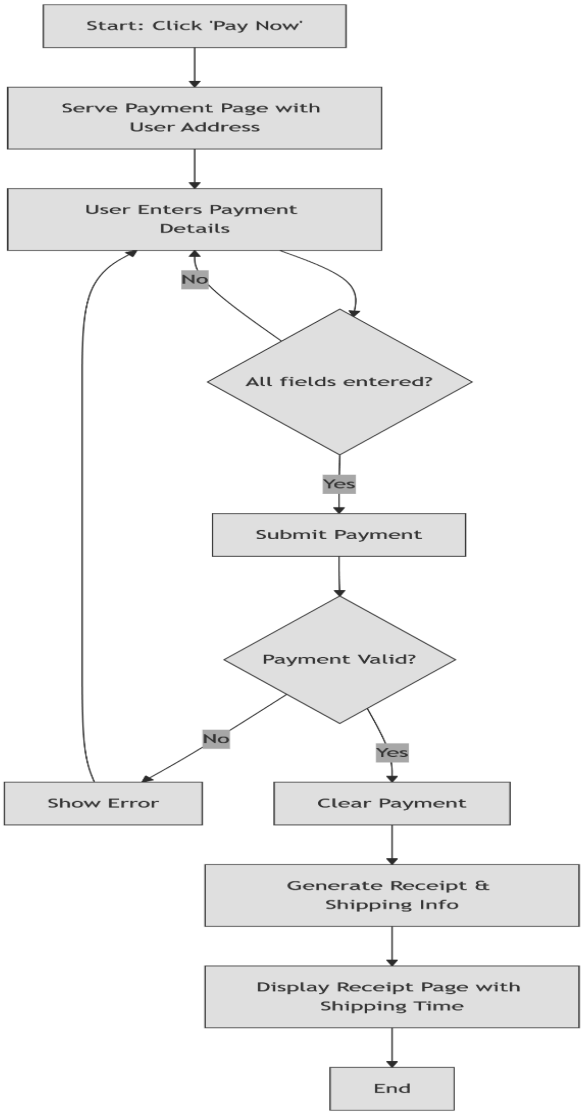
Activity Diagram for UC-PAY-1, UC-PAY-2

Legend:

Rectangle: Action/Activity

Diamond: Decision Node

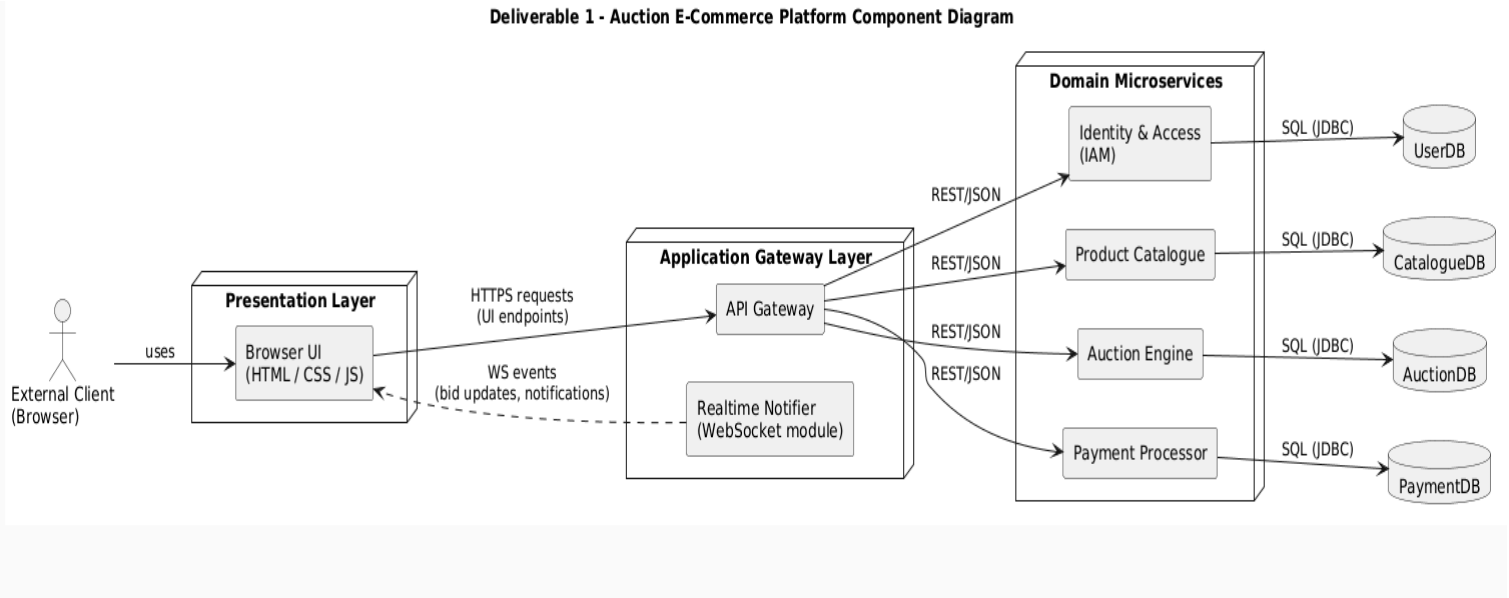
Arrows: Flow/Transition



Component Diagram

<u>Legend</u>

Stick figure (Client)	End user that is outside of the system boundary
Large labeled boxes	Subsystem boundaries
Smaller boxes instead of layers	Components/services deployed within layers.
Database cylinder icons	Data stores used by the services
Solid arrows labeled “REST/JSON”	Synchronous API calls
Lines labeled “HTTPS requests”	Browser-to-gateway communications for routes/pages that interact with the UI
Dashed arrows labeled “WS events”	Asynchronous push-style messages
SQL (JDBC) labels	Service-to-database connectivity



Architecture

Services:

Service Name	Description	Exposed Interface Names	Interface Description
Gateway Service	Façade / entry-point that forwards /api/* requests to downstream services.	GW:AuthGatewayController GW:UsersGatewayController GW:HealthController	GW:AuthGatewayController – /api/auth/* proxy routes to IAM /auth/*. GW:UsersGatewayController – /api/users/* proxy routes to IAM /users/*. GW:HealthController – /api/health for gateway status.
Identity and Authorization Management Service	Manages user sign-up, sign-in, token validation, role authorization, password reset, and user profile retrieval (shipping address).	IAM:AuthController IAM:UserController	IAM:AuthController – /auth/* endpoints for authentication, token validation, role authorization, and password reset. IAM:UserController – /users/* endpoint for user profile retrieval.
Catalogue Service	Manages the lifecycle of auction items, including validating new 'Forward Auction' listings, storing	CatalogueFacade	Provides a simplified interface for the Front-End and Gateway to search for items, retrieve

	item descriptions/shipping costs, and enabling keyword-based search functionality.		item details, and validate new seller listings. This interface will also be very important in terms of validating different functions without exposing the database logic.
Payment Service	Handles payment processing, receipt generation and shipping information for the user after the auction is won.	PaymentManagement OrderManagement ShippingManagement ReceiptManagement	PaymentManagement- Process payments securely OrderManagement- Manages order lifecycle ShippingManagement- Handles shipping information and delivery time ReceiptManagement- Delivers receipt page with details
Auction Service	Manages the auction lifecycle, such as: creating listings, processing bids, tacking highest bids, closing auctions and determining winners.	AuctionManagement BidManagement	AuctionManagement - Endpoints for creating, retrieving and closing auctions BidManagement - Endpoints for placing and validating bids.

Interfaces:

Interface Name	Operations	Operation Descriptions
GW:HealthController	Map health() → GET /api/health	Returns {status: "gateway ok"} to confirm gateway is running.

IAM:AuthController	1) signup Map<String,Object> signup(SignupRequest req) → POST /auth/signup (201 Created) 2) login AuthResponse login(LoginRequest req) → POST /auth/login (200 OK) 3) validate ValidateTokenResponse validate(String Authorization) → POST /auth/validate (200 OK) 4) forgotPassword Map<String,String> forgotPassword(ForgotPasswordRequest req) → POST /auth/password/forgot (200 OK) 5) resetPassword Map<String,String> resetPassword(ResetPasswordRequest req) → POST /auth/password/reset (200 OK)	1) POST /auth/signup (UC-IAM-01) Input: JSON SignupRequest (validated with @Valid): username, password, firstName, lastName, email, role, shippingAddress{streetNumber, streetName, city, country, postalCode}. Processing: creates user, hashes password, enforces unique username/email. Success (201): JSON { userId, username, role, message }. Failure: 400 validation errors (missing/blank fields), 409 conflict on duplicate username/email (as implemented in your service/exception handling). 2) POST /auth/login (UC-IAM-02) Input: JSON LoginRequest (validated): username, password. Processing: verifies password hash; issues JWT. Success (200): AuthResponse JSON { token, tokenType, expiresIn, userId, username, role }. Failure: 401 if credentials invalid; 400 if request invalid (missing fields). 3) POST /auth/validate (UC-IAM-04) Input: Header Authorization: Bearer <jwt> (required).

	6) authorizeRole Map<String,Object> authorizeRole(String Authorization, Role requiredRole) → GET /auth/authorize?requiredRole=... (200 OK)	Processing: extractBearer() checks header format; validates JWT signature/expiry; extracts claims. Success (200): ValidateTokenResponse JSON { valid, userId, username, role } (valid=true for good token; valid=false for invalid/expired token depending on your service logic). Failure: If header missing/doesn't start with Bearer → throws IllegalArgumentException (commonly mapped to 400 unless you map differently). 4) POST /auth/password/forgot (UC-IAM-03) Input: JSON ForgotPasswordRequest (validated): typically username (per your script). Processing: generates reset token linked to user and stores it (one-time / expirable depending on model). Success (200): JSON { message: "Password reset token generated", resetToken }. Failure: 400 if request invalid; may be 404 if username not found (depends on your service behavior). 5) POST /auth/password/reset (UC-IAM-03) Input: JSON ResetPasswordRequest (validated): resetToken, newPassword. Processing: verifies reset token (exists, not used, not

		expired), updates password hash, marks token used. Success (200): JSON { message: "Password reset successful" }. Failure: 400 invalid/missing fields; 401/404/409 depending on how you signal invalid/expired/used reset tokens (document what your service currently returns). 6) GET /auth/authorize (UC-IAM-05) Input: Header Authorization: Bearer <jwt> and query param requiredRole mapped to enum Role (required). Processing: validates JWT; compares user role claim vs requiredRole. Success (200): JSON { authorized: <true/false>, requiredRole: "<ROLE>" }. Failure: 400 if requiredRole missing/invalid enum value; 400 if Authorization header missing/invalid prefix (via extractBearer()).
IAM:UserController	UserProfileResponse getUserProfile(String userId) → GET /users/{userId}	getUserProfile – Returns profile including shipping address (used by Payment).
CatalogueFacade	List<Item> CatalogueFacade:getItemsByKeyword(String keyword)	Goes through the database for items matching the keyword and filters out expired auctions.

	int CatalogueFacade:addItem(Item DTO details)	Validates input (positive price, valid duration), calculates the End Date, and adds the new item to the DB.
	Item CatalogueFacade:getItemDetails (int itemID)	Retrieves the full details (static) (Description, Shipping Cost, Initial Price) for a single item page.
AuctionManagement	AuctionResponse createAuction(AuctionDetails details, String token) used by the Gateway.	AuctionManagement: createAuction() - Validates seller credentials via Identity Service, validates item exists via Catalogue Service, creates new auction record with calculated end time, and schedules automatic closure.
	Auction getAuction(String auctionId) used by Gateway, BidManagement.	AuctionManagement: getAuction() - Retrieves details of a specific auction, including current highest bid, status, and time remaining
	AuctionResponse closeAuction(String auctionId) used by System Scheduler.	AuctionManagement: closeAuction() - Changes auction status to "Closed", determines winner if reserve price met, notifies winner and seller, initiates payment process
	List<Auction> getActiveAuctions() used by Gateway	AuctionManagement:getActiveAuctions() - Returns list of all auctions with status "Active" and end time not yet reached
BidManagement	BidResponse placeBid(String	BidManagement: placeBid() - Validates user token

	auctionId, double bidAmount, String userId, String token) used by Gateway	via Identity Service, checks auction is active, validates bid amount is higher than current highest bid, records bid with timestamp, updates auction's current highest bid
	List<Bid> getBidHistory(String auctionId) used by Gateway	BidManagement: getBidHistory() - Returns a chronological list of all bids placed on a specific auction
	Bid getHighestBid(String auctionId) used by AuctionManagement	BidManagement: getHighestBid() - Returns the current highest bid for an auction, including bidder ID and bid amount
PaymentManagement	initiatePayment(userId, itemId, amount) validatePaymentDetails(paymentRequest) processPayment(paymentRequest) verifyPaymentStatus(transactionId)	initiatePayment() Starts the payment validatePaymentDetails() Checks if payment details are valid processPayment() Processes payment verifyPaymentStatus() Verifies payment status
OrderManagement	createOrder(userId, itemId, paymentDetails) updateOrderStatus(orderId, status) getOrderDetails(orderId)	createOrder() Creates order updateOrderStatus() Updates the status of the order getOrderDetails() Gets details of the order
ShippingManagement	getShippingTime(itemId) calculateShippingCost(itemId, address)	getShippingTime(itemId) Gets shipping time calculateShippingCost() Calculates shipping cost

ReceiptManagement	generateReceipt(orderId, paymentDetails) sendConfirmation(userId,receiptData)	generateReceipt() Generates receipt page with shipping details sendConfirmation() Sends confirmation of shipping to user

Activities Plan, Product Backlog, and Sprint Backlog

Product backlog:

EPIC A - Identity and Authorization Management

ID	Task
A1	Sign-Up (UC-IAM-01)
A2	Sign-In (UC-IAM-02)
A3	Reset Password (UC-IAM-03)
A4	Validate Session/Token (UC-IAM-04)
A5	Authorize role-based actions (UC-IAM-05)
A6	Make user profile retrievable (UC-IAM-06)

EPIC B - Catalogue Service

ID	Task
B1	DB Schema Design (UC-CAT-7)
B2	Item Creation API (UC-CAT-7)
B3	Keyword Search API (UC-CAT-2.1)
B4	Item Detail API (UC-CAT-2)
B5	Expiration Filtering Logic (UC-CAT-2.1)
B6	Containerization (System Wide)
B7	AI Description Generator (UC-CAT-7)

EPIC C - Auction Service

ID	Task
C1	DB Schema Design for Auctions and Bids (UC1-AS, UC2-AS)
C2	Create Auction API (UC1-AS)
C3	Place Bid API (UC2-AS)

C4	Get Active Auctions API (UC4-AS)
C5	Bid Validation Logic (UC2-AS)
C6	Auction Closure Scheduler (UC3-AS)
C7	Determine Winner Logic (UC3-AS)
C8	Real-time Bid Notification (Pub-Sub Integration)

EPIC D - Payment Service

ID	Task
D1	DB Schema Design
D2	Payment API (UC-PAY-1)
D3	Order API (UC-PAY-1,UC-PAY-2)
D4	Shipping API (UC-PAY-1, UC-PAY-2)
D5	Receipt API (UC-PAY-2)
D6	Error Handling (UC-PAY-1)
D7	Receipt Sent Notification (Pub-Sub Integration)

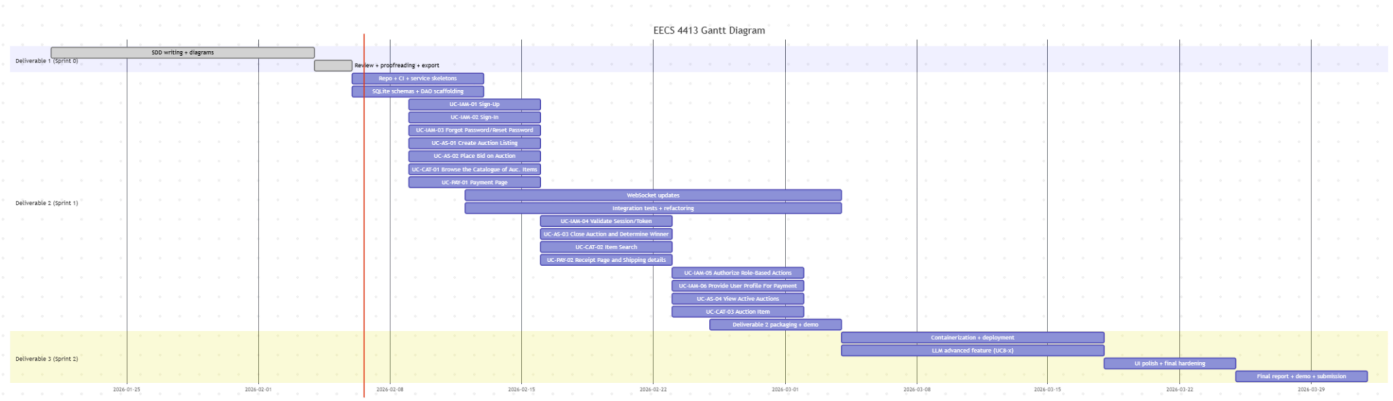
GANTT diagram:

(The diagram may be unclear due to its large size. Therefore, I have provided a Google Drive link to the PNG for better resolution.

https://drive.google.com/file/d/1aAVKnE-WW3yjkkAI6CozNTGCzwK8xUE7/view?usp=sharing)

<u>Legend</u>

Horizontal axis	Project timeline
Rows	Deliverable/sprint phases
Task bars	Scheduled tasks
Bar length	Task duration



Group meeting logs: Group met on January 30th to discuss who is completing what service, project logistics, and overall next steps/steps of completion. The group decided to create a WhatsApp group chat as the primary means of communication to coordinate logistics and update each other on project progress.

Anton will complete the catalogue service, Ravneet will complete the Identity and Authorization Management (IAM) Service, Syed will complete the Auction service, and Peyton will complete the Payment Service.

Meeting time: 20 minutes. Future meetings/communication for Deliverable 1 will be via WhatsApp call/texting. Team met again online on March 31st to do a final testing of all use cases, making sure that all services are running on the frontend/backend without any errors before final submission.

Test Driven Development

<u>Ravneet Test Cases</u>

(Ravneet Test Case 1)

Test ID	TC-GW-IAM-01
Category	IAM (via Gateway) – Sign-Up
Requirements Coverage	UC-IAM-01 Sign-Up (Success)
Initial Condition	Gateway and IAM are running; username/email do not already exist.
Procedure	1. Send POST /api/auth/signup with valid user + shipping address + role. 2. Observe HTTP status and response body.
Expected Outcome	HTTP **200** with JSON including message, userId, username, and role.
Notes	Confirms gateway forwarding works and IAM persists user.

(Ravneet Test Case 2)

Test ID	TC-GW-IAM-02
Category	IAM (via Gateway) – Sign-Up
Requirements Coverage	UC-IAM-01 Sign-Up (Duplicate Username/Email)
Initial Condition	A user already exists with the same username and email from TC-GW-IAM-01.

Procedure	1. Send the same POST /api/auth/signup payload again. 2. Observe status.
Expected Outcome	HTTP **409 Conflict** with JSON error message (e.g., “Username already exists”).
Notes	Verifies uniqueness enforcement and proper error propagation through gateway.

(Ravneet Test Case 3)

Test ID	TC-GW-IAM-03
Category	IAM (via Gateway) – Sign-In
Requirements Coverage	UC-IAM-02 Sign-In (Success)
Initial Condition	User exists (created in TC-GW-IAM-01).
Procedure	1. Send `POST /api/auth/login` with correct username/password. 2. Capture returned token.
Expected Outcome	HTTP 200 with JSON containing token, expiresIn, userId, username, role.
Notes	Token is used in later tests as Authorization: Bearer <token>.

(Ravneet Test Case 4)

Test ID	TC-GW-IAM-04

Category	IAM (via Gateway) – Sign-In
Requirements Coverage	UC-IAM-02 Sign-In (Failure – Wrong Password)
Initial Condition	User exists (created in TC-GW-IAM-01).
Procedure	1. Send POST /api/auth/login with correct username but wrong password. 2. Observe response.
Expected Outcome	HTTP 401 Unauthorized with JSON error message (e.g., “Invalid username or password”).
Notes	Confirms authentication rejection and error status forwarding.

(Ravneet Test Case 5)

Test ID	TC-GW-IAM-05
Category	IAM (via Gateway) – Token Validation
Requirements Coverage	UC-IAM-04 Validate Session/Token (Valid)
Initial Condition	Valid token obtained from TC-GW-IAM-03.
Procedure	1. Send POST /api/auth/validate with header Authorization: Bearer <token>. 2. Observe JSON result.
Expected Outcome	HTTP 200 with JSON: valid=true, and correct userId/username/role.

Notes	Confirms token is accepted by IAM and identity claims are returned.

(Ravneet Test Case 6)

Test ID	TC-GW-IAM-06
Category	IAM (via Gateway) – Token Validation
Requirements Coverage	UC-IAM-04 Validate Session/Token (Invalid Token)
Initial Condition	Gateway and IAM running.
Procedure	1. Send POST /api/auth/validate with header Authorization: Bearer not.a.real.token. 2. Observe JSON result.
Expected Outcome	HTTP 200 with JSON: valid=false, and null identity fields.
Notes	This is your current design choice (return 200 with valid=false). Some systems return 401; either is acceptable if consistent and documented.

(Ravneet Test Case 7)

Test ID	TC-GW-IAM-07
Category	IAM (via Gateway) – Role Authorization
Requirements Coverage	UC-IAM-05 Authorize Role-Based Actions (Authorized)
Initial Condition	Valid token for a BUYER user from TC-GW-IAM-03.

Procedure	1. Send GET /api/auth/authorize?requiredRole=BUYER with Authorization: Bearer <token>. 2. Observe response.
Expected Outcome	HTTP 200 with JSON: authorized=true and requiredRole=BUYER.
Notes	Confirms role checking works.

(Ravneet Test Case 8)

Test ID	TC-GW-IAM-08
Category	IAM (via Gateway) – Role Authorization
Requirements Coverage	UC-IAM-05 Authorize Role-Based Actions (Not Authorized)
Initial Condition	Valid token for BUYER user.
Procedure	1. Send GET /api/auth/authorize?requiredRole=SELLER with same BUYER token. 2. Observe response.
Expected Outcome	HTTP 200 with JSON: authorized=false and requiredRole=SELLER.
Notes	Confirms negative authorization case.

(Ravneet Test Case 9)

Test ID	TC-GW-IAM-09
Category	IAM (via Gateway) – Input Validation

Requirements Coverage	UC-IAM-01 Sign-Up (Validation Failure)
Initial Condition	Gateway and IAM running.
Procedure	1. Send POST /api/auth/signup with missing required field(s) (script uses missing password). 2. Observe response.
Expected Outcome	HTTP 400 Bad Request with validation JSON identifying the missing field (e.g., password must not be blank).
Notes	Confirms server-side validation + friendly error payload.

(Ravneet Test Case 10)

Test ID	TC-GW-IAM-10
Category	Gateway (IAM proxy) – Request Parameter Handling
Requirements Coverage	UC-IAM-05 Authorize Role-Based Actions (Bad Request)
Initial Condition	Gateway and IAM running; token exists (script attempts with token).
Procedure	1. Send GET /api/auth/authorize without requiredRole query param. 2. Observe response.
Expected Outcome	HTTP 400 Bad Request indicating missing required parameter.
Notes	If your response currently includes a large stack trace, you can clean it by setting server.error.include-stacktrace=never in

	gateway properties.

(Ravneet Test Case 11)

Test ID	TC-GW-IAM-11
Category	IAM via Gateway — User Profile Retrieval
Requirements Coverage	UC-IAM-06 Provide User Profile For Payment
Initial Condition	User has been created (sign-up succeeded) and a valid userId exists.
Procedure	1. GET /api/users/{userId} using the userId returned by signup/login. 2. Observe HTTP response and JSON body.
Expected Outcome	200 OK with JSON containing userId, username, email, firstName, lastName, role, and shippingAddress fields.
Notes	This is the endpoint Payment will use to populate shipping address on the payment page.

(Ravneet Test Case 12)

Test ID	TC-GW-IAM-12
Category	IAM via Gateway — Forgot Password
Requirements Coverage	UC-IAM-03 Forgot Password / Reset Password (Token Generation)
Initial Condition	User exists in the database (created in earlier steps).

Procedure	1. POST /api/auth/password/forgot with body { "username": "<username>" }. 2. Observe response.
Expected Outcome	200 OK with JSON containing "message": "Password reset token generated" and a resetToken.
Notes	Returning resetToken is suitable for demos/tests. In a real system this would be emailed, not returned in response.

(Ravneet Test Case 13)

Test ID	TC-GW-IAM-13
Category	IAM via Gateway — Reset Password
Requirements Coverage	UC-IAM-03 Forgot Password / Reset Password (Reset Success)
Initial Condition	A valid resetToken was generated from TC-GW-12 and is not expired/used.
Procedure	1. POST /api/auth/password/reset with body { "resetToken": "<token>", "newPassword": "<newPassword>" }. 2. Observe response.
Expected Outcome	200 OK with JSON { "message": "Password reset successful" }.
Notes	Confirms the reset token is accepted and password hash is updated.

(Ravneet Test Case 14)

Test ID	TC-GW-IAM-14
Category	IAM via Gateway — Sign-In (After Password Reset)
Requirements Coverage	UC-IAM-02 Sign-In (Success after password change)
Initial Condition	Password reset completed successfully (TC-GW-13).
Procedure	1. POST /api/auth/login with username and the new password. 2. Observe response.
Expected Outcome	200 OK with JSON containing a new token, expiresIn, userId, username, and role.
Notes	Confirms password reset actually took effect.

<u>Anton Test Cases</u>

(Anton Test Case 1)

Test ID	TC-CAT-01
Category	Database Search
Requirements Coverage	UC 2.1 - Item Search
Initial Condition	System is running, Catalogue DB contains a record with status 'active'.
Procedure	1. Navigate to the Search bar. 2. Enter desired string 3. Click the Search button.

Expected Outcome	The system returns a result list containing the desired string with its initial price and end date.
Notes	Tests the keyword matching logic, and ensures retrieval of static catalogue data.

(Anton Test Case 2)

Test ID	TC-CAT-02
Category	Catalogue - Item Management
Requirements Coverage	UC 7 - Auction Item (Static Catalogue)
Initial Condition	User is logged in as a Seller, and the Catalogue DB is connected.
Procedure	1. Enter description. 2. Set Start Price 3. Set Duration: exactly 48 hours 4. Click 'Post Auction'.
Expected Outcome	A new entry is created in the DB with an End Date exactly 48 hours from the current time.
Notes	Verifies the auction setup, shipping cost storage, and End Date calculation.

(Anton Test Case 3)

Test ID	TC-CAT-03

Category	Catalogue - Business Logic
Requirements Coverage	UC 2 - Display Items (Filtering)
Initial Condition	The database contains an ‘expired item’ where the End_Date has passed.
Procedure	1. User performs a general search or browses the "All Items" category.
Expected Outcome	The expired item is not included in the returned list of items.
Notes	Ensures the Catalogue only serves items whose time hasn't expired.

(Anton Test Case 4)

Test ID	TC-CAT-04
Category	Catalogue - Input Validation
Requirements Coverage	UC 7 - Auction Item (Validation)
Initial Condition	User is on the 'Sell Item' page.
Procedure	1. Enter Description: "Test". 2. Enter Start Price: -10 (Or any negative value). 3. Click 'Submit'.
Expected Outcome	The system prevents submission and displays an error: "Initial price must be a positive integer."

Notes	Ensures that auction items must be sold at positive integers.

Mustafa Test Cases

(Mustafa Test Case 1)

Test ID	TC-AUC-01
Category	Auction Controller — Create Auction
Requirements Coverage	UC1-AS: Create Auction Listing (Success) AuctionControllerTest
Initial Condition	AuctionService mocked; valid JWT token in Authorization header; valid request body with positive duration and reserve price.
Procedure	Send POST /api/auctions with Authorization: Bearer token, itemId, sellerId, durationHours=24, reservePrice=99.99.
Expected Outcome	HTTP 201 Created; response body contains auctionId="AUC_1" and status="ACTIVE".
Notes	Confirms controller correctly delegates to service and returns 201 with full auction JSON.

(Mustafa Test Case 2)

Test ID	TC-AUC-02
Category	Auction Controller — Input Validation
Requirements Coverage	UC1-AS: Create Auction Listing (Validation Failure)

	AuctionControllerTest
Initial Condition	Valid token present; request body contains invalid values (durationHours=-1, reservePrice=0).
Procedure	Send POST /api/auctions with the invalid JSON body.
Expected Outcome	HTTP 400 Bad Request; auctionService never invoked.
Notes	Confirms Jakarta @Valid annotations reject invalid input before reaching service logic.

(Mustafa Test Case 3)

Test ID	TC-AUC-03
Category	Auction Controller — Close Expired
Requirements Coverage	UC3-AS: Close Auction and Determine Winner AuctionControllerTest
Initial Condition	AuctionService mocked to return 3 for closeExpiredAuctions().
Procedure	Send POST /api/auctions/close-expired.
Expected Outcome	HTTP 200 OK; response body contains the string "Closed 3 expired auctions".
Notes	Confirms manual trigger endpoint for the scheduler-driven closure flow.

(Mustafa Test Case 4)

Test ID	TC-AUC-04
Category	Bid Controller — Place Bid (Success)
Requirements Coverage	UC2-AS: Place Bid on Auction (Success) BidControllerTest
Initial Condition	BidService mocked; valid token; request body contains bidderId and bidAmount=150.00.
Procedure	Send POST /api/auctions/AUC_1/bids with Authorization header and JSON body.
Expected Outcome	HTTP 200 OK; response JSON contains bidId="BID_1", auctionId="AUC_1", bidderId="BUYER_1", success=true.
Notes	Confirms the full bid placement controller path with correct response shape.

(Mustafa Test Case 5)

Test ID	TC-AUC-05
Category	Bid Controller — Input Validation
Requirements Coverage	UC2-AS: Place Bid on Auction (Validation Failure) BidControllerTest
Initial Condition	Valid token; request body contains empty bidderId and negative bidAmount=-1.
Procedure	Send POST /api/auctions/AUC_1/bids with the invalid body.

Expected Outcome	HTTP 400 Bad Request; bidService never invoked.
Notes	Verifies that @Valid on PlaceBidRequest prevents invalid bids from reaching service logic.

(Mustafa Test Case 6)

Test ID	TC-AUC-06
Category	Bid Controller — Bid History
Requirements Coverage	UC4-AS: View Active Auctions / Bid history retrieval BidControllerTest
Initial Condition	BidService mocked to return two bids for auction AUC_1.
Procedure	Send GET /api/auctions/AUC_1/bids.
Expected Outcome	HTTP 200 OK; response is a JSON array of length 2 with correct bidId values.
Notes	Confirms bid history endpoint returns chronological list.

(Mustafa Test Case 7)

Test ID	TC-AUC-07
Category	Bid Controller — No Bids
Requirements Coverage	UC2-AS: Place Bid — Highest Bid (No Bids Case) BidControllerTest

Initial Condition	BidService mocked to return null for getHighestBid("AUC_1").
Procedure	Send GET /api/auctions/AUC_1/highest-bid.
Expected Outcome	HTTP 204 No Content.
Notes	Confirms the no-content response when an auction has not yet received any bids.

(Mustafa Test Case 8)

Test ID	TC-AUC-08
Category	Auction Service — Create Auction
Requirements Coverage	UC1-AS: Create Auction Listing (Service Logic) AuctionServiceTest
Initial Condition	Token validates to SELLER_1; IAM mock confirms SELLER role; Catalogue mock confirms item exists; DAO mock assigns auctionId="AUC_1".
Procedure	Call auctionService.createAuction(request, token).
Expected Outcome	Response has auctionId="AUC_1", status=ACTIVE, sellerId derived from token (not request body), currentHighestBid=0, non-null startTime and endTime.
Notes	Verifies that sellerId comes from token validation, not the request body — a key security guarantee.

(Mustafa Test Case 9)

Test ID	TC-AUC-09
Category	Auction Service — Close with Winner
Requirements Coverage	UC3-AS: Close Auction and Determine Winner (Reserve Met) AuctionServiceTest
Initial Condition	Auction exists, highest bid ($150) exceeds reserve price ($100); DAO and payment mocks configured.
Procedure	Call auctionService.closeAuction(auctionId).
Expected Outcome	Response has status=CLOSED, winnerId="BUYER_1"; serviceClient.initiatePayment() called exactly once with correct amount.
Notes	Confirms winner determination and payment initiation in the same closure flow.

(Mustafa Test Case 10)

Test ID	TC-AUC-10
Category	Auction Service — Close without Winner
Requirements Coverage	UC3-AS: Close Auction — Reserve Not Met (No Sale) AuctionServiceTest
Initial Condition	Auction exists; highest bid ($150) is below reserve price ($200).
Procedure	Call auctionService.closeAuction(auctionId).

Expected Outcome	Response has status=NO_SALE, winnerId=null; serviceClient.initiatePayment() is never called.
Notes	Confirms that payment is not triggered when the reserve price is not met.

(Mustafa Test Case 11)

Test ID	TC-AUC-11
Category	Bid Service — Invalid Token
Requirements Coverage	UC2-AS: Place Bid — Authentication Failure BidServiceTest
Initial Condition	IAM mock returns valid=false for the provided token.
Procedure	Call bidService.placeBid("AUC_1", request, "bad-token").
Expected Outcome	RuntimeException thrown; no database operations performed.
Notes	Confirms the service immediately rejects bids with invalid tokens before any DAO calls.

(Mustafa Test Case 12)

Test ID	TC-AUC-12
Category	Bid Service — Closed Auction
Requirements Coverage	UC2-AS: Place Bid — Auction Not Active BidServiceTest

Initial Condition	Valid token; auction exists with status=CLOSED.
Procedure	Call bidService.placeBid("AUC_CLOSED", request, token).
Expected Outcome	AuctionClosedException thrown.
Notes	Confirms bids are rejected on closed auctions regardless of bid amount.

(Mustafa Test Case 13)

Test ID	TC-AUC-13
Category	Bid Service — Bid Too Low
Requirements Coverage	UC2-AS: Place Bid — Bid Amount Validation BidServiceTest
Initial Condition	Valid token; active auction with currentHighestBid=$200; bid request has bidAmount=$200 (equal, not strictly greater).
Procedure	Call bidService.placeBid("AUC_BID_TOO_LOW", request, token).
Expected Outcome	InvalidBidException thrown.
Notes	Confirms the strict greater-than rule, equal bids are rejected.

(Mustafa Test Case 14)

Test ID	TC-AUC-14
Category	Repository — Auction DAO (Integration)

Requirements Coverage	UC1-AS, UC4-AS: Data persistence and active auction retrieval AuctionRepositoryTest
Initial Condition	Fresh SQLite test database; one ACTIVE auction with end time in the far future; one ACTIVE auction already expired; one CLOSED auction.
Procedure	Call auctionDAO.findActiveAuctions().
Expected Outcome	Returns exactly one auction — the non-expired ACTIVE one. Expired and closed auctions are excluded.
Notes	Integration test against the real SQLite schema. Verifies both status and end-time filtering logic.

(Mustafa Test Case 15)

Test ID	TC-AUC-15
Category	Repository — Bid DAO (Integration)
Requirements Coverage	UC2-AS: Bid ordering and highest bid retrieval BidRepositoryTest
Initial Condition	Fresh SQLite test database; auction with three bids — two tied at $200 placed at different times, one lower at $150.
Procedure	Call bidDAO.findHighestBidByAuctionId(auctionId).
Expected Outcome	Returns the bid with bidderId="BUYER_3" — highest amount ($200), and among ties, the most recent timestamp wins.

Notes	Integration test verifying the tie-breaking rule (highest amount → newest timestamp). Critical for correct winner determination.

(Mustafa Test Case 16)

Test ID	TC-LB-01
Category	Leaderboard Controller — Weekly Leaderboard
Requirements Coverage	UC-LB-01: Weekly Leaderboard (Success) LeaderboardControllerTest
Initial Condition	LeaderboardService mocked to return one entry with bidderId="BUYER001", bidAmount=500.0, rank=1.
Procedure	Send GET /api/leaderboard.
Expected Outcome	HTTP 200 OK; response contains count=1, entries[0].rank=1, entries[0].bidderId="BUYER001", entries[0].bidAmount=500.0, and a generatedAt timestamp.

(Mustafa Test Case 17)

Test ID	TC-LB-02
Category	Leaderboard Controller — Empty Leaderboard
Requirements Coverage	UC-LB-01: Weekly Leaderboard (No Bids This Week) LeaderboardControllerTest
Initial Condition	LeaderboardService mocked to return an empty list.

Procedure	Send GET /api/leaderboard.
Expected Outcome	HTTP 200 OK; count=0; entries is an empty JSON array.

(Mustafa Test Case 18)

Test ID	TC-LB-03
Category	Leaderboard Controller — Bidder Stats
Requirements Coverage	UC-LB-03: Per-Bidder Weekly Stats (Success) LeaderboardControllerTest
Initial Condition	LeaderboardService mocked to return one entry for BUYER001 with bidAmount=500.0.
Procedure	Send GET /api/leaderboard/bidder/BUYER001.
Expected Outcome	HTTP 200 OK; bidderId="BUYER001", bidCount=1, bids[0].bidAmount=500.0, highestBid=500.0.

(Mustafa Test Case 19)

Test ID	TC-LB-04
Category	Leaderboard Controller — Bidder Stats (No Activity)
Requirements Coverage	UC-LB-03: Per-Bidder Weekly Stats (No Bids) LeaderboardControllerTest
Initial Condition	LeaderboardService mocked to return empty list for BUYER999.

Procedure	Send GET /api/leaderboard/bidder/BUYER999.
Expected Outcome	HTTP 200 OK; bidderId="BUYER999", bidCount=0.

(Mustafa Test Case 20)

Test ID	TC-LB-05
Category	Leaderboard Controller — Weekly Stats
Requirements Coverage	UC-LB-02: Weekly Statistics Aggregate LeaderboardControllerTest
Initial Condition	getWeeklyBidCount() returns 5; getWeeklyLeaderboard() returns one entry with bidAmount=500.0.
Procedure	Send GET /api/leaderboard/stats.
Expected Outcome	HTTP 200 OK; weeklyBidCount=5, topBidsCount=1, highestBid=500.0, generatedAt present.

(Mustafa Test Case 21)

Test ID	TC-LB-06
Category	Leaderboard Controller — Historical Lookup
Requirements Coverage	UC-LB-04: Historical Leaderboard by Week LeaderboardControllerTest
Initial Condition	getWeeklyLeaderboard(LocalDateTime) mocked to return one entry.

Procedure	Send GET /api/leaderboard/week/2026/12.
Expected Outcome	HTTP 200 OK; year=2026, week=12, count=1.

(Mustafa Test Case 22)

Test ID	TC-LB-07
Category	Leaderboard Controller — Invalid Period
Requirements Coverage	UC-LB-02: Highest Bid Query — Invalid Period Input LeaderboardControllerTest
Initial Condition	getHighestBidForPeriod("MONTH") throws IllegalArgumentException("Unsupported period").
Procedure	Send GET /api/leaderboard/highest?period=MONTH.
Expected Outcome	HTTP 400 Bad Request; response body contains error field.
Notes	Confirms that only the "WEEK" period is accepted; invalid periods are rejected with a descriptive error.

(Mustafa Test Case 23)

Test ID	TC-LB-08
Category	Leaderboard Service — Add Entry
Requirements Coverage	UC-LB-01: Bid Entry Ingestion LeaderboardServiceTest

Initial Condition	Repository mock configured; no prior entries.
Procedure	Call leaderboardService.addBidEntry("AUC001", "ITEM001", "BUYER001", "John", 500.0, "SELLER001", "Jane").
Expected Outcome	leaderboardRepository.save() called exactly once with a LeaderboardEntry containing the provided fields.
Notes	Confirms the entry-creation path triggered by the Auction Service after each successful bid.

(Mustafa Test Case 24)

Test ID	TC-LB-09
Category	Leaderboard Service — Top Bidder Aggregation
Requirements Coverage	UC-LB-02: Top Bidders with Activity Metrics LeaderboardServiceTest
Initial Condition	Repository mock returns a TopBidderProjection for BUYER001 with bidCount=3, highestBid=500, totalBidValue=1200.
Procedure	Call leaderboardService.getTopBiddersForPeriod("WEEK", 5).
Expected Outcome	Returns one TopBidderResponse with bidderId="BUYER001", rank=1, correct bid metrics.

(Mustafa Test Case 25)

Test ID	TC-LB-10

Category	Leaderboard Service — Invalid Limit Rejected
Requirements Coverage	UC-LB-02: Input Validation LeaderboardServiceTest
Initial Condition	Service running; limit parameter of 0 provided.
Procedure	Call leaderboardService.getTopBiddersForPeriod("WEEK", 0).
Expected Outcome	IllegalArgumentException thrown; repository never queried.
Notes	Ensures the service enforces a minimum limit of 1 to prevent meaningless queries.

<u>Peyton Test Cases</u>

(Peyton Test Case 1)

Test ID	TC-PAY-01
Category	Payment Service - Complete Payment
Requirements Coverage	UC-PAY-1, UC-PAY-2
Initial Condition	Auction Is won by the user.
Procedure	Click Pay now Enter Payment details Click submit Wait for payment processing
Expected Outcome	Payment is processed successfully, and the user is redirected to the receipt page.

(Peyton Test Case 2)

Test ID	TC-PAY-02
Category	Payment Service - Missing Field
Requirements Coverage	UC-PAY-1, UC-PAY-2
Initial Condition	Auction Is won by the user
Procedure	Click Pay now Enter Payment details other than cvv Click submit
Expected Outcome	Payment is not processed successfully and an error message is displayed indicating that all fields must be filled with valid information. The user remains on the payment page. No receipt is generated.

(Peyton Test Case 3)

Test ID	TC-PAY-03
Category	Payment Service - Invalid Card information
Requirements Coverage	UC-PAY-1,UC-PAY-2
Initial Condition	Auction Is won by the user
Procedure	1. Click Pay now 2. Enter invalid payment details

	3. Click submit
Expected Outcome	Payment starts to process but an error message appears stating that the payment is declined check card details. The user stays on the payment page. No receipt is generated.

(Peyton Test Case 4)

Test ID	TC-PAY-04
Category	Payment Service - Receipt Generation and Display
Requirements Coverage	UC-PAY-1,UC-PAY-2
Initial Condition	Payment successfully processed. Shipping cost $5.99, and the item cost $15.00
Procedure	1. Payment successful 2. Receipt page generates
Expected Outcome	All valid information is displayed, including amount of days shipping will take and the total amount paid of $20.99, a sum of shipping and item cost.

(Peyton Test Case 5)

Test ID	TC-PAY-05
Category	Payment Service - Invalid cvv
Requirements Coverage	UC-PAY-1,UC-PAY-2

Initial Condition	Auction is won by user
Procedure	4. Click Pay now 5. Enter invalid payment details 6. Click submit
Expected Outcome	Payment is not processed successfully and an error message is displayed indicating that cvv must be 3 digits. The user remains on the payment page. No receipt is generated.

(Peyton Test Case 6)

Test ID	TC-PAY-06
Category	Payment Service - Invalid expiry data
Requirements Coverage	UC-PAY-1,UC-PAY-2
Initial Condition	Auction is won by user
Procedure	7. Click Pay now 8. Enter invalid payment details 9. Click submit
Expected Outcome	Payment is not processed successfully and an error message is displayed indicating that expiry date must be in the format xx/xx with a valid month. The user remains on the payment page. No receipt is generated.

(Peyton Test Case 7)

Test ID	TC-PAY-07
Category	Payment Service - Wrong user
Requirements Coverage	UC-PAY-1,UC-PAY-2
Initial Condition	Auction isnt won by user but tries to pay
Procedure	10. Click Pay now 11. Enter valid payment details 12. Click submit
Expected Outcome	Payment is not processed successfully and an error message is displayed indicating only winner can pay

Acknowledgements

AI was used in this deliverable to assist with brainstorming use case scenarios, giving feedback on UML diagrams according to course conventions, and proofreading technical documentation according to the success criteria. All design decisions, architectural choices, and implementations were made independently by the team members. AI-generated suggestions were critically evaluated and modified to meet project requirements.

AI Usage for IAM Service

- AI for this service was used primarily as a tool for offering suggestions when it came to industry best practices, aided with debugging when errors appeared during development, and helped with test development. During development, when the question of how the server will verify the client's identity appeared, AI was consulted on the different methods that one can use to achieve this. JWT design was one of the methods offered. AI was used to give an explanation and examples of how one implements JWT into a project. Using the examples provided, I developed an idea of how I wanted to structure the format of my token for the JWT that I was implementing into the IAM service. Once I created my token, I had AI evaluate it to see what improvements I can make to it. Through the examples AI had provided, I decided on how I would want to deliver the token to the client, and what protected routes will require a user to provide their token. AI was used when simulating suggestions for different ways to persist data as the service is running. AI aided in the process of developing a global exception handler class for the IAM service. It helped with generating an example on how to format an exception handler for my service. I used that example as a format on how to create the remaining exception handlers. AI helped with showcasing examples on how to generate a gateway service to function as a facade. It gave me an idea on how to create the gateway service and connect it to the other services. Due to limited

bash experience, AI was helpful in the development of test cases to check if the service's endpoints and features were functioning properly. In conclusion, AI was a major asset in the speed of iteration, showcasing industry best practices for web-development, and aiding in troubleshooting problems. A limitation that was seen by AI use was that suggestions can be version-sensitive, as import packages can vary between Spring versions. Therefore, one import that is not supported by the versions of the pre-existing packages can create errors. Used it to help develop the layout of a page on the frontend, used the principles learned to help me develop the remaining frontend pages. Helped with the implementation of essential libraries for storing a user's login information across pages, without having the user needing to log each time they switch pages. Helped with implementing a css file to style the pages for aesthetic purposes. Aided with following good design practices to avoid code reuse.

AI Usage for Payment Service

- AI was used to first help get an idea what the structure of this service should look like. Once it looked good I then used ai to generate the project. I had to manually fix some of the code myself and I also consulted the AI to fix issues I saw pop up. What ended up happening was I got put into a loop and the AI couldn't fix the issue so I had to figure out the issue myself. AI was then used to help extend my service to communicate with the others and help generate code to do so. It also helped me with testing to make sure the code worked and pinpoint why it wasn't working. Overall all is very useful for generating a lot of code very fast but you end up spending a lot of time debugging and fixing errors so it's important to understand the code the ai generated. Ai was also used to help generate postman script for testing.

AI Usage for Catalogue Service

- AI was primarily used to conduct file structure implementation, and how I could implement the different designs without overcomplicating the intended use of design patterns. Code was then manually written and refined to meet test requirements. AI also saved me lots of time when it came to searching different documents/other students' endpoints and

variables, describing what components do what, thus ensuring swift and efficient progress in service production. It also saved me lots of time when it came to debugging, finding out what functions should do what, and how to actually implement the test cases pertaining to the use cases when all the other different services are also involved. AI was used to look for the backend mismatches, as certain services were previously not communicating with each other as expected. It identified the issues and suggested possible fixes to help fix mismatches and fix the HATEOAS implementation that was not integrated from Deliverable 2.

AI Usage for Auction Service

- AI was mainly used as a development aid during the implementation of the Auction service. It provided suggestions on how the service could be structured and what endpoints should exist for handling auction related actions such as creating auctions, placing bids, and retrieving auction data. AI was also useful when debugging issues that appeared during development, particularly with request handling and service logic. In addition, AI was used to generate example implementations for validating auction rules, such as ensuring that a new bid is higher than the current highest bid and that bids are only accepted while an auction is active. These examples were used as references and then adapted to fit the specific requirements of the project. AI also helped with generating and understanding test cases to verify that endpoints behaved correctly. Overall, AI helped speed up development and debugging, but the generated code still required manual review and adjustments to ensure it worked correctly within the project.

AI Usage for Leaderboard Service

- AI was primarily used as a development aid during the implementation of the leaderboard service, with a stronger focus on debugging and frontend integration. It provided guidance on structuring the service and handling data aggregation for ranking users based on auction activity. A significant portion of its usefulness came during debugging, where it helped identify issues related to data inconsistencies, incorrect sorting logic, and edge cases in

ranking calculations. Additionally, AI assisted with the frontend aspects of the service by suggesting ways to display leaderboard data clearly and efficiently, including formatting, updating views dynamically, and ensuring smooth communication between the frontend and backend. It also helped generate example scenarios to test how the leaderboard would respond to different bidding activities. As with the auction service, these suggestions and code examples were used as references and refined to meet the project's specific requirements, with manual verification ensuring correctness and consistency.

Tools used for the development include IDEs like Vscode, Eclipse, Github and DockerDesktop.

Tools used for the sequence, activity and component diagrams were as follows:

https://sequencediagram.org

https://lucid.app/lucidchart

https://www.umlet.com

https://app.diagrams.net/

https://plantuml.com/

And the document/report was compiled on Google Docs.